VoiceAI – Comprehensive Speech AI System Documentation

---

1. Project Title

VoiceAI: An End-to-End Speech AI System for Automatic Speech Recognition, Speaker Diarization, Text-to-Speech, Voice Cloning, and Voice Conversion

---

2. Objective

The primary objective of this project is to design and develop a **modular Speech AI system** capable of processing audio inputs and performing multiple intelligent tasks, including:

- Automatic Speech Recognition (ASR)
- Speaker Diarization
- Text-to-Speech (TTS)
- Voice Cloning
- Voice Conversion

The system aims to simulate real-world applications such as:

- AI voice assistants
- Meeting transcription systems
- Call center analytics
- Accessibility tools for speech-impaired users

---

3. Problem Statement

In real-world scenarios, audio data often contains:

- Multiple speakers
- Background noise
- Unstructured conversational flow

Traditional systems fail to:

- Accurately separate speakers
- Provide structured transcripts
- Offer voice transformation capabilities

This project addresses these challenges by integrating multiple AI techniques into a single unified system.

---

4. System Overview

The system is implemented as a **web-based application using Streamlit**, allowing users to interact with different speech processing modules.

Core Functionalities

1. Audio transcription with timestamps
2. Multi-speaker identification
3. Speaker-wise transcript segmentation
4. Speech generation from text
5. Voice cloning using pitch estimation
6. Voice conversion using signal processing

---

5. System Architecture

Pipeline Flow

Audio Input
↓
Preprocessing (Resampling, Mono Conversion)
↓
ASR (Whisper Model)
↓
Feature Extraction (MFCC)
↓
Clustering (K-Means)
↓
Speaker Diarization
↓
Transcript-Speaker Alignment
↓
User Interface Display
↓
Optional Modules:
→ Text-to-Speech

→ Voice Cloning
→ Voice Conversion

---

6. Detailed Module Explanation

6.1 Audio Input Module

The system supports:

- File upload (WAV, MP3, FLAC, OGG)
- Live microphone recording

The input is temporarily stored and passed for preprocessing.

6.2 Audio Preprocessing

Before processing, audio is standardized:

- Converted to **16 kHz sampling rate**
- Converted to **mono channel**

This ensures:

- Consistent model input
- Reduced computational complexity
- Better ASR accuracy

Implementation:
Handled using audio utilities such as pydub and librosa

6.3 Automatic Speech Recognition (ASR)

The ASR module uses the **Whisper model**.

Key Features of Whisper

- Transformer-based deep learning model
- Supports multilingual transcription
- Provides timestamped segments
- Works offline

Processing Steps

1. Load model based on size (tiny, base, small, medium)
2. Pass audio input
3. Generate:
    - Text segments
    - Start and end timestamps
    - Language detection

Implementation:
Defined in ASR module

Important Note

Whisper internally uses **PyTorch**, satisfying the requirement of deep learning usage.

6.4 Speaker Diarization

Speaker diarization identifies “who spoke when”.

Steps Involved

1. Feature Extraction

- Extract **MFCC (Mel-Frequency Cepstral Coefficients)**
- Compute:
    - Mean
    - Standard deviation
    - Delta features

2. Feature Normalization

- StandardScaler is applied to normalize features

3. Clustering

- K-Means clustering is used to group similar voice patterns

4. Automatic Speaker Detection

- Silhouette score is used to estimate optimal number of speakers

5. Post-processing

- Smoothing applied using majority voting
- Merge consecutive segments

Implementation:
Diarization pipeline

6.5 Transcript and Speaker Alignment

- Each ASR segment has timestamps
- Diarization segments also have timestamps
- The system:
    - Computes midpoint of each ASR segment
    - Assigns speaker based on matching diarization segment

This ensures accurate mapping of:
Text → Speaker

6.6 Text-to-Speech (TTS)

Two backends are implemented:

1. gTTS (Online)

- Uses Google TTS API
- Outputs MP3 format

2. pyttsx3 (Offline)

- Uses system TTS engine
- Outputs WAV format

Fallback Mechanism

- System first tries offline engine
- Falls back to gTTS if unavailable

6.7 Voice Cloning Module

This module approximates speaker voice characteristics.

Method Used

- Estimate **Fundamental Frequency (F0)** using:
    - librosa.pyin

Steps

1. Extract pitch from reference audio
2. Generate speech using TTS
3. Match pitch using:
    - Log-scale frequency conversion
4. Apply pitch shift

Limitation

- Captures pitch only
- Does not model:
    - Accent
    - Timbre
    - Emotion

6.8 Voice Conversion Module

Transforms one voice into another.

Modes

1. Target-based conversion:
    - Matches pitch with target audio
2. Manual conversion:
    - User controls:
        - Pitch (semitones)
        - Speed (time-stretch)

Techniques Used

- Pitch shifting
- Time stretching

---

7. User Interface

Developed using **Streamlit**

Features:

- Interactive tabs
- Audio playback
- Colored transcripts
- Speaker statistics
- File download support

---

8. Technologies Used

Programming Language

- Python

Frameworks and Libraries

- PyTorch (via Whisper)
- Librosa (audio processing)
- NumPy (numerical computation)
- Scikit-learn (clustering)
- Streamlit (UI)
- SoundFile, pydub (audio handling)

---

9. Machine Learning and Deep Learning Concepts

Machine Learning

- K-Means clustering
- Silhouette score optimization

Deep Learning

- Transformer architecture (Whisper)

Audio Processing

- MFCC feature extraction

- Pitch estimation (F0)
- Spectrogram-related processing

---

10. System Design Strengths

- Modular architecture
- Offline capability
- Multi-feature integration
- Lightweight implementation (CPU-friendly)
- Real-world applicability

---

11. Limitations

- K-Means is less accurate than deep learning diarization models
- No support for overlapping speech
- Voice cloning is pitch-based only
- No real-time streaming support

---

12. Future Enhancements

- Integrate Pyannote for deep learning diarization
- Use neural TTS models (Tacotron, VITS)
- Add real-time streaming transcription
- GPU acceleration
- Cloud deployment (AWS)

---

13. Key Interview Points

- Whisper uses PyTorch internally
- MFCC captures frequency characteristics of speech
- K-Means groups speakers based on voice similarity
- Silhouette score helps determine optimal clusters

- Pitch shifting uses logarithmic frequency scaling

---

14. Conclusion

VoiceAI is a comprehensive Speech AI system that combines:

- Automatic Speech Recognition
- Speaker Diarization
- Speech Synthesis
- Voice Transformation

The project demonstrates strong understanding of:

- AI/ML fundamentals
- Audio signal processing
- System integration
- Practical implementation of real-world AI applications